GUÍA PRÁCTICA DE STRATA

Guía de Mac

Instalación y Primeros Experimentos

Paso a paso desde cero, sin saber programar



cd ~/sirisys · 24 · 57 · 6 · 12 · 16 · same ledger: YES

Juan A. Pérez-Gómez

Versión 13.5 · 13 de noviembre de 2026

Interpretación independiente · sin afiliación con SIRISYS

Índice

Paso 1 · Descargar el proyecto 1
Paso 2 · Abrir la Terminal 2
Paso 3 · Comprobar si tienes Python 2
Paso 4 · Situarte en la carpeta 3
Paso 5 · Instalar las piezas 3
Paso 6 · Comprobar que todo funciona (gratis, sin modelo) 3
Paso 7 · Elegir el modelo 4
Paso 8 · La prueba corta del informe 4
Paso 9 · Experimentos sugeridos 5
Paso 10 · Tus resultados 6
Si algo falla 7
Resumen 7

Guía paso a paso desde cero para STRATA, una interpretación independiente en software de SIRISYS (versión 13.5). No hace falta saber programar ni haber abierto nunca una Terminal.

Antes de empezar

Vas a necesitar:

- Tu Mac y conexión a internet.
- El repositorio STRATA de GitHub: github.com/juanan-prz/STRATA (el paso 1 explica cómo descargarlo).
- Para los experimentos con modelo: Ollama, gratis y local (recomendado), o una clave de API de Anthropic (opcional). Las pruebas de funcionamiento no necesitan ninguno de los dos.

La primera vez son unos 30 minutos de instalación. Después, cada sesión empieza en un minuto.

REGLA DE ORO

Los comandos hay que copiarlos y pegarlos tal cual, sin cambiar nada. Un espacio de más o una tilde donde no toca y no funciona. Copiar y pegar es lo correcto, no hacer trampa.

Guía de una interpretación independiente en software. STRATA no es una obra oficial de SIRISYS ni implica afiliación con SIRISYS.

Paso 1 Descargar el proyecto

- Abre en Safari https://github.com/juanan-prz/STRATA.
- Pulsa el botón verde **Code** y después **Download ZIP**.
- Abre el ZIP descargado (doble clic): aparece una carpeta.
- Muévela a tu **carpeta de inicio** (la que lleva tu nombre de usuario y el icono de una casa) y cámbiale el nombre a exactamente `sirisys` (minúsculas, sin acentos ni espacios).

ATENCIÓN

No la pongas en Documentos ni en el Escritorio. Si iCloud los sincroniza con «Optimizar almacenamiento», puede dejar los archivos solo en la nube; con la Wi-Fi apagada, los programas no pueden leerlos y la prueba sale vacía sin avisar.

Dentro deben estar, juntos y sin subcarpetas, estos archivos (entre otros):

Archivo	Para qué sirve
`Sirisys_Framework_v13_5.py`	El sistema.
`Sirisys_Server.py` · `sirisys_loader.py`	Arrancan el modo experimento, con la vista en vivo.
`Sirisys_Live_Visualizer.html`	La vista en vivo del campo.
`Sirisys_Static_Visualizer.html`	El inspector: examina después una instancia guardada.
`run_report.py`	Ejecuciones medidas: escribe un informe con sus resultados.
`field_topology.py` · `sirisys_measures.py`	Mediciones que usan los informes. Vigilan también que el modelo responda.
`test_*.py` · `sirisys_testkit.py`	Pruebas de funcionamiento: cinco archivos y lo que comparten.
`measure_phase3.py` · `live_c2.py` · `soak.py`	Mediciones: criterios, perceptos reales y vida larga.
`requirements.txt`	La lista de piezas que hay que instalar.

Descarga siempre desde el ordenador, no desde el móvil. Si los archivos pasan por el portapapeles de un teléfono o por un editor de texto enriquecido, se corrompen las comillas y el código deja de funcionar.

Cuando haya una versión nueva, cambia el nombre de la carpeta antigua (por ejemplo, `sirisys_v134`) antes de descargar la nueva: así conservas sus resultados y no se mezclan los archivos.

Paso 2 Abrir la Terminal

La Terminal es una ventana donde se escriben órdenes en vez de hacer clic. Pulsa Command + barra espaciadora, escribe Terminal y pulsa Enter. Se abre una ventana con un cursor parpadeando. No puedes romper nada escribiendo ahí: si un comando no existe, simplemente te lo dice.

Paso 3 Comprobar si tienes Python

Copia esto en la Terminal y pulsa Enter:

```
python3 --version
```

Si responde Python 3.10 o superior (por ejemplo 3.12.4), ya lo tienes: salta al paso 4. Si dice `command not found`, te ofrece instalar herramientas o la versión es más antigua: ve a python.org/downloads, pulsa el botón amarillo Download Python, instálalo como cualquier programa, cierra la Terminal, ábrela de nuevo y repite el comando.

Paso 4 Situarte en la carpeta

```
cd ~/sirisys
ls
```

El primero entra en la carpeta; el segundo lista lo que hay dentro. Debe responder con los archivos del paso 1. Si los ves, vas bien.

Paso 5 Instalar las piezas

```
pip3 install -r requirements.txt
```

Verás mucho texto pasando: es normal y tarda un par de minutos. Cuando vuelvas a poder escribir, ha terminado. Si sale un aviso en rojo que dice `externally-managed-environment` (suele pasar con el Python de Homebrew), usa esta otra línea:

```
pip3 install --break-system-packages -r requirements.txt
```

Paso 6 Comprobar que todo funciona (gratis, sin modelo)

Lanza estas cinco pruebas, una a una. No gastan dinero ni necesitan internet:

Comando	Qué debe decir al final
`python3 test_sirisys.py`	`SUMMARY: 24/24 passed` (≈ 15 s)
`python3 test_phase1.py`	`SUMMARY: 57/57 passed` (≈ 20 s)
`python3 test_phase2_guarantee.py`	`SUMMARY: 6/6 passed` (≈ 5 s)
`python3 test_phase4.py`	`SUMMARY: 12/12 passed` (≈ 5 s)
`python3 test_ontology.py`	`SUMMARY: 16/16 passed` (≈ 5 s)

Si alguna línea dice `FAIL`, anota qué prueba falló y las líneas que la preceden; vuelve a descargar el ZIP desde el ordenador y repite, porque un archivo dañado es la causa más común. Para ver el campo en vivo:

```
python3 Sirisys_Server.py --play
```

Abre en Safari http://127.0.0.1:8000. Para pararlo, vuelve a la Terminal y pulsa Control + C: la instancia se guarda en la carpeta `sirisys_runs` y no se borra nada. Para examinarla después, abre `Sirisys_Static_Visualizer.html` en Safari y elige el `state.json` de su carpeta.

Paso 7 Elegir el modelo

El modelo es opcional y solo traduce: pone nombres legibles a lo que el campo produce y formula una pregunta en cada acercamiento al Sombunal. Lo que dice nunca cambia lo que hace el campo: una vida con modelo deja el mismo registro que sin él.

Opción A — Ollama (recomendada). El modelo corre dentro de tu Mac: gratis, sin enviar nada fuera.

- Descárgalo de ollama.com e instálalo como cualquier programa. Queda en marcha en segundo plano, con un icono en la barra de arriba.
- En la Terminal, descarga un modelo. Por ejemplo:

```
ollama pull llama3.1
ollama list
```

`ollama list` debe mostrar el modelo descargado; si responde con un error, Ollama no está en marcha: ábrelo. Usa siempre el mismo modelo en las ejecuciones que quieras comparar.

Opción B — Anthropic (opcional, solo para experimentos). En console.anthropic.com crea una cuenta, ve a API Keys → Create Key, copia la clave (empieza por `sk-ant-`; solo se muestra una vez) y añade saldo en Billing sin activar la recarga automática. La suscripción a Claude no da saldo de API: son cosas separadas. Después, en la Terminal:

```
export ANTHROPIC_API_KEY="sk-ant-aqui-tu-clave"
```

No responde nada: eso significa que ha ido bien. Dura mientras la ventana esté abierta. No guardes la clave en ningún archivo de la carpeta.

Paso 8 La prueba corta del informe

```
python3 run_report.py --cycles 20 --seeds 1 --backend ollama --profile minimal
```

(Con Anthropic, cambia `ollama` por `anthropic`.) Antes de empezar, el programa hace tres llamadas reales al modelo. Fíjate en:

Qué buscar	Qué debe decir
Las 16 líneas de comprobación del principio	Todas con `[ok ]`.
`model probe`	ok, y los segundos que tarda cada llamada con el contexto de una ejecución. Si el modelo no responde o responde vacío, el programa se para aquí con `STOP` y dice por qué (mira «Si algo falla»).
`sample names`	Dos nombres solos, no frases. «SYSTEM_AM» es un nombre; «The emergent construct is SYSTEM_AM.» es una frase.
Al final: `model calls`	`failures 0` y `empty 0`.
Al final: `same ledger in every arm`	`YES`: el campo deja el mismo registro con el modelo y sin él.

Paso 9 Experimentos sugeridos

Una secuencia corta, con el perfil `minimal` (sin narración ni hora del día), en este orden. Delante de los largos va `caffeinate -i`: impide que el Mac se duerma mientras dura la prueba y deja de actuar por sí solo al terminar. No cierres la ventana de la Terminal; en un portátil, déjalo enchufado y con la tapa abierta.

1 · El control, sin modelo (unos segundos). El campo con tres flujos de perceptos sintéticos: comprueba que cumple sus criterios en tu máquina. Al final, la línea `criteria` debe decir `PASS` en los tres; si alguno dice `FAIL`, repítelo con el Mac en reposo, porque una de las medidas es el tiempo que tarda cada ciclo.

```
python3 run_report.py --backend none --profile minimal --cycles 1000 --seeds 3
mv sirisys_report.json control_report.json
```

El segundo comando cambia el nombre del informe para que no lo pise el siguiente.

2 · Perceptos reales, sin modelo (≈ 10 minutos). El campo con los perceptos de tu Mac durante una vida larga: comprueba que se sostiene con lo que percibe de verdad. El resultado queda en el archivo (paso 10). El campo percibe el procesador y la memoria: cierra antes todas las aplicaciones (también Ollama) y no uses el Mac mientras corre.

```
caffeinate -i python3 live_c2.py 10000 none minimal > live_none.json
```

3 · Perceptos reales con el modelo, y su repetición sin él (de media hora a dos horas). Comprueba que lo que dice el modelo no cambia lo que hace el campo. Con Ollama en marcha y nada más abierto. Al empezar dice en la Terminal cuánto tardará como máximo.

```
caffeinate -i python3 live_c2.py 100 ollama minimal > live_ollama.json
python3 live_c2.py 100 none minimal --replay live_ollama.json > live_replay.json
```

El segundo vuelve a vivir la misma vida, con las mismas percepciones, sin modelo, y tarda unos segundos. Al terminar debe decir `same ledger as the replayed run: YES`.

Mientras corre `live_c2.py` no se ve nada más en pantalla: es normal, escribe el resultado al terminar. Si el modelo deja de responder a mitad (Ollama se cierra o se cuelga), el programa se detiene solo y lo dice en la Terminal con `STOPPED`. Pon Ollama en marcha y vuelve a lanzar ese mismo comando; el archivo interrumpido no hace falta.

Opcional: ver el campo en vivo con el modelo.

```
export SIRISYS_LLM_BACKEND=ollama
python3 Sirisys_Server.py --play --profile minimal
```

Abre http://127.0.0.1:8000 como en el paso 6. Al pasar por un constructo se ve el nombre que le dio el modelo y, debajo, su clave, la del campo. Arriba se ve el modelo; si deja de responder, el campo se pausa, la cabecera lo marca en rojo (`watch tripped`) y el registro de eventos dice por qué. Nunca a la vez que una prueba: páralo con Control + C antes de lanzar la siguiente.

Paso 10 Tus resultados

En la carpeta `sirisys` habrán aparecido cuatro archivos:

- `control_report.json`, el control;
- `live_none.json`, los perceptos reales sin modelo;
- `live_ollama.json`, los perceptos reales con el modelo;
- `live_replay.json`, su repetición sin modelo.

Son archivos de texto (JSON): cada uno recoge la configuración de la ejecución, los criterios de autoajuste con su veredicto, la salud del modelo, los tiempos y la cabeza del ledger. Los que llevan modelo guardan además cada nombre que dio, con su respuesta entera. La guía técnica (`SIRISYS_Guide_v13_ES.md`) explica qué contiene cada uno. Para comparar resultados entre máquinas o modelos, anota también:

- el nombre del modelo de Ollama que usaste (lo dice `ollama list`);
- el chip y la memoria de tu Mac (menú Apple → Acerca de este Mac).

Si algo falla

Lo que ves	Qué hacer
`command not found: python3`	Python no está instalado: vuelve al paso 3.
`No such file or directory`	No estás en la carpeta o faltan archivos: repite el paso 4 y comprueba con `ls`.
`externally-managed-environment`	Usa la segunda línea del paso 5.
Alguna prueba dice `FAIL`	Vuelve a descargar el ZIP desde el ordenador y repite. Si sigue fallando, anota la prueba y su mensaje.
`model active: False · STOP: … not available`	Ollama no está en marcha (`ollama list` da error: ábrelo y repite), el modelo no está descargado o, con Anthropic, la clave no se puso en esta ventana.
`STOP: … answered empty`	El modelo «piensa» antes de responder y se queda sin espacio para el nombre. Usa un modelo general, por ejemplo `llama3.1` (paso 7).
`STOPPED — model health`	El modelo dejó de responder a mitad. Pon Ollama en marcha y vuelve a lanzar ese comando.
`same ledger …: NO`	Es un error del código, no un resultado: guarda la salida y avisa en GitHub.
El archivo .json pesa 0 bytes	La carpeta está en Documentos o en el Escritorio, sincronizada con iCloud: muévela a la carpeta de inicio (paso 1) y repite.
`authentication_error / 401 · credit balance is too low`	Con Anthropic: la clave está mal copiada (cópiala entera, sin espacios) o falta saldo en console.anthropic.com → Billing.
`already open by another writer`	Ya hay otro SIRISYS escribiendo en esa carpeta: cierra la otra ventana de Terminal o pulsa Control + C en ella.
`Address already in use`	El modo experimento ya está en marcha en otra ventana de la Terminal: páralo allí con Control + C.
La prueba se cortó al dormirse el Mac	Vuelve a lanzarla con `caffeinate -i` delante; en un portátil, enchufado y con la tapa abierta.
La ventana parece parada	Normal: está trabajando. Si de verdad se colgó, Control + C.

Resumen

Cada sesión empieza en la carpeta, y conviene comprobar que todo funciona:

```
cd ~/sirisys
python3 test_sirisys.py
python3 test_phase1.py
python3 test_phase2_guarantee.py
python3 test_phase4.py
python3 test_ontology.py
```

La secuencia de experimentos del paso 9:

```
python3 run_report.py --backend none --profile minimal --cycles 1000 --seeds 3
mv sirisys_report.json control_report.json
caffeinate -i python3 live_c2.py 10000 none minimal > live_none.json
caffeinate -i python3 live_c2.py 100 ollama minimal > live_ollama.json
python3 live_c2.py 100 none minimal --replay live_ollama.json > live_replay.json
```

Fin del documento · Versión 13.5 · 13 de noviembre de 2026